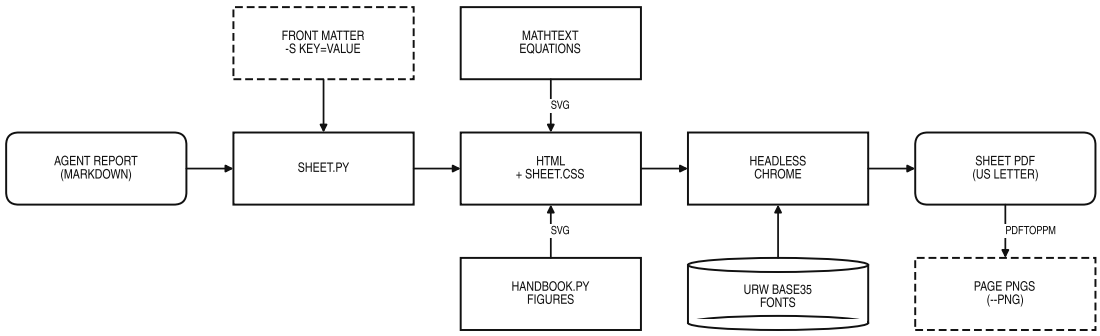
APOLLO STYLE REVIEW SHEETS

Reference and specimen sheet. Every block the format has, with its source beside it. Start a new sheet from template.md.

A.1 FUNCTIONAL DESCRIPTION.

A review sheet is a section of a handbook made from one agent's report: numbered paragraphs, figures and tables with drawing numbers, call-outs, checklists and equations, printed in black on US Letter to be read with a pen. The report is plain Markdown; `sheet.py` renders it as it is, and the `:::` blocks below add the handbook furniture. Figures are drawn by `handbook.py` as line art with live lettering, equations by matplotlib's mathtext, and the page by headless Chrome from HTML and `sheet.css`.



APO-501A

Figure A-1. Review Sheet Functional Diagram

```
python sheet.py report.md                          writes report.pdf beside it
python sheet.py report.md -o out.pdf --png         and page PNGs, to look without a printer
python sheet.py notes.md -s section=3 -s tab=PHY   someone else's report, without editing it
```

A.2 PAGE DESIGN.

Table A-1. Design Rules

Element	Set in	Rule
Body	Nimbus Roman 9.6 pt on 1.32	justified and hyphenated; the first line after a heading indented 0.3 in
Lettering on figures	Nimbus Sans Narrow, 5 to 7 pt	capitals only; the same face live in the PDF, never outlines
Checklists, code	Nimbus Mono PS 8.3 pt	a checklist is boxed, under a black bar carrying its title and role
Page	US Letter	margins 0.95 in top, 0.85 bottom, 0.45 left, 0.3 right; a 0.95 in gutter for numbers
Ink	black only	white paper to print; `paper: cream` for a screen copy
Headings	## capitals, ### underlined	the paragraph number hangs in the gutter: A.4, A.4.2
Numbers	Figure A-1, Table A-1, (A-1)	figures and tables count through the section; table titles above, captions below
Drawing numbers	APO-501A ...	one per figure, bottom right, from the tab
Strokes	0.6 to 0.9 pt	dashed for optional or planned, hatched for used or selected, solid black for on
Series	dash pattern, end label	solid, dashed, dotted, dash-dot; labelled at the line's end, never by colour or legend
Header, footer	doc, handbook, system; mission, dates, page	dates MM.DD.YYYY; page A-n from the section
Changes	bar in the gutter	`:::changed` beside anything altered after the basic date; set `change:`

The page is quiet on purpose: no colour, no shading but hatch, no rules but the table frames and the `---` line. Anything that needs colour to be read needs redrawing, not colouring; a colour screen capture prints inverted to ink with `gray=1`.

A.3 TEXT AND CALL-OUTS.

A.3.1 Headings and paragraphs.

A `#` heading names the sheet when there is no `title`; a later `#` starts a part on a new page. `##`, `###` and `####` are numbered from `section`, and any number the report gave its own headings is dropped. Inline marks: `**bold**`, `*italic*`, `_underline_`, backticks for code, `^sup^`, `~sub~`, `$math$`, and `--` for a dash. Links keep their words; paper has no links. A line starting `<!--` opens a comment that never prints, up to `-->`.

A.3.2 Call-outs and checklists.


```
:::caution
Text that, if ignored, costs
time or data.
:::

:::warning
Text that, if ignored, damages
hardware or loses work for good.
:::

:::note
Anything worth a box that is
neither.
:::

:::checklist Render a Sheet | Any
1  python sheet.py r.md --png
2  read every page PNG
3  fix, render again
:::

:::changed
A paragraph altered after the
basic date.
:::
```

CAUTION

Text that, if ignored, costs time or data.

WARNING

Text that, if ignored, damages hardware or loses work for good.

NOTE

Anything worth a box that is neither.

RENDER A SHEET	ANY
1	python sheet.py r.md --png
2	read every page PNG
3	fix, render again

A paragraph altered after the basic date. The bar is in the gutter, beside the paragraph number.

A.4 MATHEMATICS.

A.4.1 Inline and display.

Inline math sits on the baseline of the text: the corner of an RC low-pass is $f_c = 1/(2\pi RC)$, written `$f_c = 1/(2\pi R C)$`. A display, `$$ ... $$` on its own lines, is centred and numbered from the section:

$$|H(f)| = \frac{1}{\sqrt{1 + (f/f_c)^{2n}}} \tag{A-1}$$

where f = frequency *Hz*
f_c = corner frequency, where the gain is down 3 dB *Hz*
n = order of the Butterworth filter

The `:::symbols` block above is the handbook's "where" list: one `SYMBOL | MEANING | UNIT` a line, the symbol in TeX without dollars.

A.4.2 Aligned equations.

`:::equations` sets one equation a line, aligned on its first relation (or on `&` where a line has one). A line that starts with a relation continues the one above and takes no number. Here, the length of an audio block:

```
:::equations
T_{\mathrm{block}} = N / f_s
= 128 / 44100\ \mathrm{s}
\approx 2.902\ \mathrm{ms}
G_{\mathrm{dB}}(f_c) = {-10} \log_{10} 2
\approx {-3.01}\ \mathrm{dB}
:::
```

$$T_{\mathrm{block}} = N/f_s \tag{A-2}$$
$$= 128/44100 \text{ s}$$
$$\approx 2.902 \text{ ms}$$
$$G_{\mathrm{dB}}(f_c) = -10\log_{10} 2 \tag{A-3}$$
$$\approx -3.01 \text{ dB}$$

A.4.3 A result, boxed.

`$$\boxed{...}$$` frames the line a reader should carry away:

$$\boxed{f_c = \frac{1}{2\pi RC}} \tag{A-4}$$

CAUTION

Equations are set by matplotlib's mathtext, a subset of TeX: fractions, roots, sub- and superscripts, Greek, `\left( \right)`, accents, `\mathrm`, `\mathbf`, `\mathcal`, sums and integrals. It has no environments (`\begin{aligned}`, matrices, `cases`): use `:::equations` for alignment, a table for a matrix. `\text{}` is rewritten to `\mathrm{}` and `\sqrt x` to `\sqrt{x}`. What will not parse prints as its TeX in typewriter, so nothing is lost: `\begin{pmatrix} a & b \end{pmatrix}`.

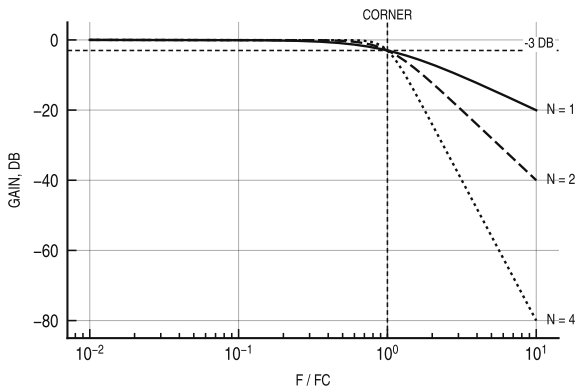
A.5 FIGURES.

Every figure is line art with a caption below and a drawing number at its lower right. `:::diagram`, `:::timeline`, `:::sequence`, `:::signals` and `:::budget` run the width of the column; a `:::plot` is 3.4 by 2.2 in unless it says `size wide` or `size W H`, so two sit side by side in a `::::row`.

A.5.1 Plots: exact functions and tabulated series.

`fn "EXPR"` draws a curve exactly from a numpy expression in `x` (`sin exp sqrt log10 pi where clip` and the like), over `range LO HI`. `y COLUMN` draws a column of a CSV against `x COLUMN`; `dots` sets it as points, the way measurements are told from a model. Thresholds are `hline`, events `mark`.

```
:::plot Butterworth Gain re the Corner
range 0.01 10
fn "-10*log10(1+x**2)" "N = 1"
fn "-10*log10(1+x**4)" "N = 2"
fn "-10*log10(1+x**8)" "N = 4"
log x
xlabel "f / fc"
ylabel "gain, dB"
ylim -85 5
hline -3.01 "-3 DB"
mark 1 "CORNER"
:::
```

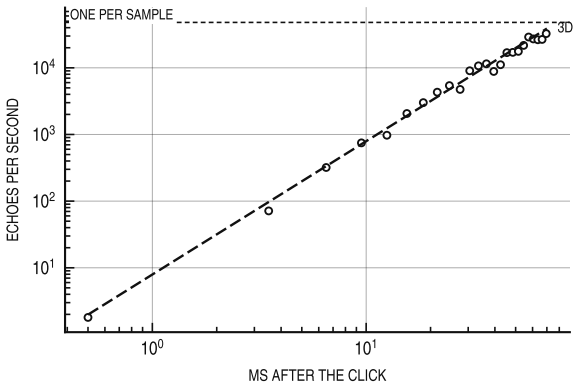


APO-502A

Figure A-2. Butterworth Gain re the Corner

```
:::plot Echo Density, 4 m Cube
csv density.csv
x ms "ms after the click"
y d3 "" dots every=6
fn "4*pi*343**3*(x/1000)**2/4**3" "3D"
log xy
ylabel "echoes per second"
hline 48000 "ONE PER SAMPLE"
:::
```

The dots are a CSV of counts; the line is $4\pi c^3 t^2/V$ for c = 343 m/s and V = 64 m^3, drawn over the CSV's own x.



APO-503A

Figure A-3. Echo Density, 4 m Cube: CSV (Dots) and Formula (Line)

APO

A.5.2 Block diagrams.

Figure A-1 is a `:::diagram`. Nodes sit on a grid of columns and rows (`grid DX DY` in inches, if the default 1.55 by 0.78 crowds the lettering); edges route themselves straight, with one elbow, or with a Z through the gap between rows or columns, whichever clears the other boxes, and are labelled on their longest run. Shapes: `box`, `round` for ends, `dash` for the optional, `drum` for storage, `text` for bare lettering; `fence "LABEL" ids...` draws a dashed subsystem boundary.

```
:::diagram Review Sheet Functional Diagram
grid 1.45 0.8
node fm "FRONT MATTER\n-s KEY=VALUE" 1 0 dash
node rep "AGENT REPORT\n(MARKDOWN)" 0 1 round
node sheet "SHEET.PY" 1 1
node fonts "URW BASE35\nFONTS" 3 2 drum
edge rep sheet
edge mt html "SVG"
edge pdf png "PDFTOPPM"
...
:::
```

A.5.3 Timelines.

Rows of bars on one axis, `hatch`, `open` or `solid`; `mark` for a moment across every row, `call` for a leader to lettering below the axis. Apollo 11, from launch:

```
:::timeline Apollo 11, Ground Elapsed Time
axis "ground elapsed time, hours" 0 200 ticks=25
bar "Earth orbit" 0 2.74 solid
bar "Translunar coast" 2.74 75.83 open
bar "Lunar orbit (CSM)" 75.83 135.40 hatch
bar "LM on the surface" 102.76 124.37 hatch
bar "EVA" 109.13 111.65 solid
bar "Transearth coast" 135.40 195.31 open
mark 102.76 "LANDING" dash
mark 195.31 "SPLASHDOWN" dash
:::
```

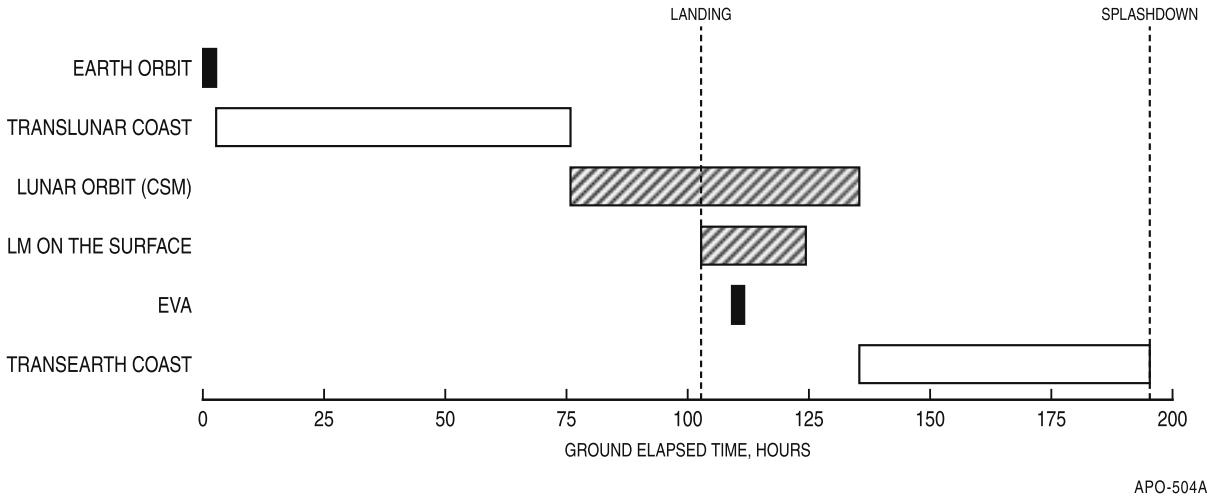
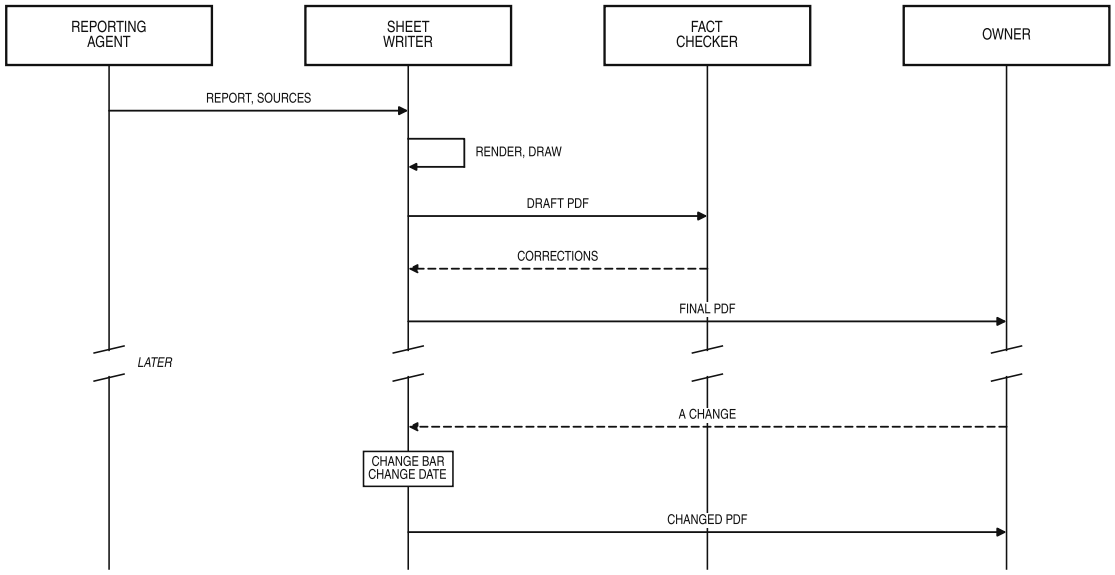


Figure A-4. Apollo 11, Ground Elapsed Time (Launch 07.16.1969, 13:32 UTC)

A.5.4 Sequence charts.

Messages between actors, one step a line, top to bottom: protocols, install flows, a review's hand-offs. `dash` for a reply, the same actor twice for its own work, `gap` where time passes. A review's hand-offs:

```
:::sequence A Review's Hand-offs
actor agent "REPORTING\nAGENT"
actor writer "SHEET\nWRITER"
actor check "FACT\nCHECKER"
actor owner "OWNER"
msg agent writer "REPORT, SOURCES"
msg writer writer "RENDER, DRAW"
msg writer check "DRAFT PDF"
msg check writer "CORRECTIONS" dash
msg writer owner "FINAL PDF"
gap "LATER"
msg owner writer "A CHANGE" dash
note writer "CHANGE BAR\nCHANGE DATE"
msg writer owner "CHANGED PDF"
:::
```



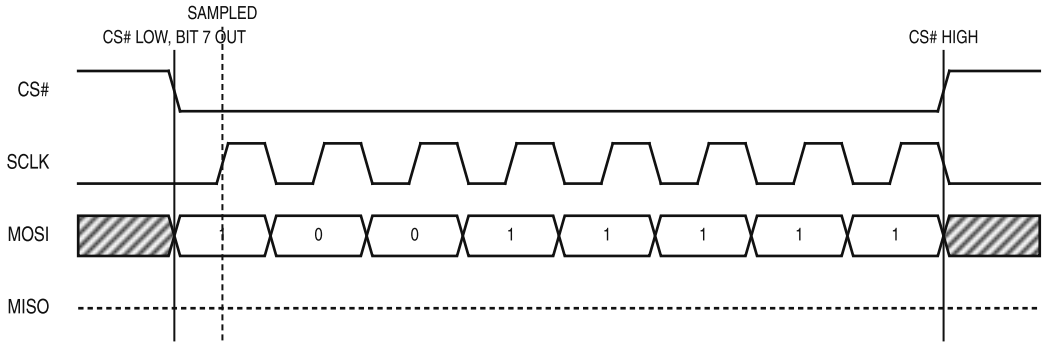
APO-505A

Figure A-5. A Review's Hand-offs

A.5.5 Timing diagrams.

`:::signals` draws lines and buses on one step axis, a character a step: `0 1` levels, `p n` a clock period starting high or low, `.` hold, `z` released, `x` unknown, `=` a bus value lettered from a list. An SPI flash taking the JEDEC read-ID command 0x9F in mode 0, shifted on the falling edge and sampled on the rising:

```
:::signals SPI Mode 0, Command 0x9F (Read JEDEC ID)
signal "CS#" 1000000001
signal "SCLK" 0nnnnnnnn0
signal "MOSI" x========x "1,0,0,1,1,1,1,1"
signal "MISO" zzzzzzzzzz
mark 1 "CS# LOW, BIT 7 OUT"
mark 1.5 "SAMPLED" dash
mark 9 "CS# HIGH"
:::
```



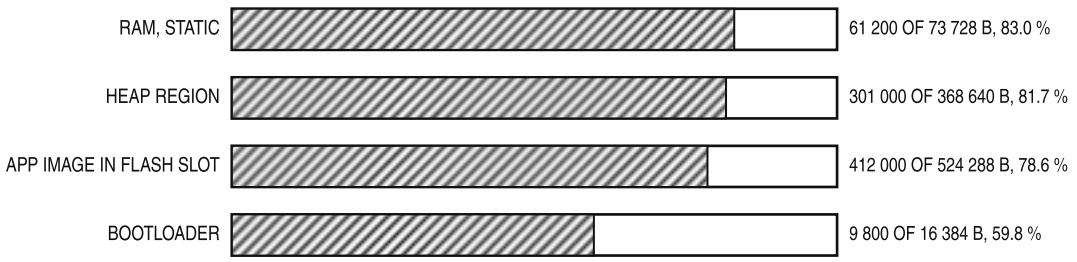
APO-506A

Figure A-6. SPI Mode 0, Command 0x9F (Read JEDEC ID)

A.5.6 Budget gauges.

Used against capacity, one gauge a line, hatched to the used fraction, with the figures and the percentage at the end; hex is read as hex. `mark PERCENT "LABEL"` sets a limit across them all. A firmware image's memory:

```
:::budget Firmware 1.0 Memory
bar "RAM, static" 61200 73728 B
bar "Heap region" 301000 368640 B
bar "App image in flash slot" 412000 0x80000 B
bar "Bootloader" 9800 0x4000 B
:::
```



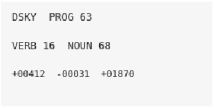
APO-507A

Figure A-7. Firmware 1.0 Memory

A.5.7 Pictures and columns.

`![Caption](path){width=half}` takes an SVG, inlined so its lettering stays live, or a PNG; `width=third` and `gray=1` (a dark screen capture, printed inverted as ink on paper) are the options. `::::row` sets its contents side by side, columns split on a line `|||`, closed by `::::`; anything can go in a column.

```
![A Screen Capture](../kitchen-
sink/screen.png){width=third gray=1}
```



APO-508A

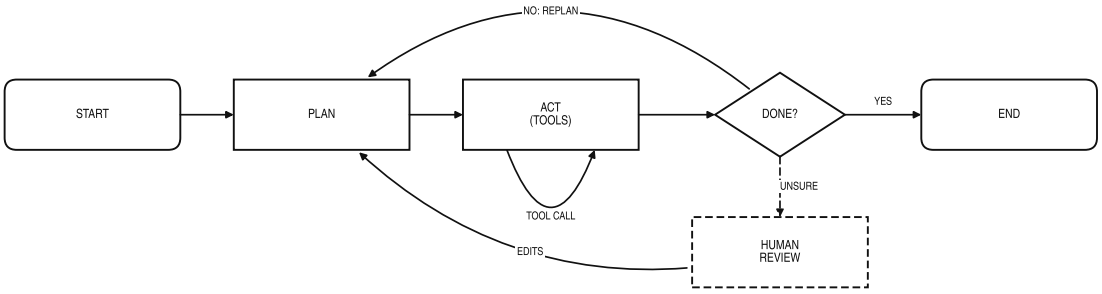
Figure A-8. A Dark Screen Capture, Printed as Ink

A.6 MORE FIGURES.

A.6.1 Graphs: decisions, back-edges and loops.

`:::diagram` draws graphs as well as block diagrams: a `diamond` node for a decision, `bend=R` on an edge for a curve (a back-edge to an earlier step; R above 0 bows right of the way it runs, below 0 left), and `loop ID` for a node that goes back to itself. An agent's loop, the shape of a LangGraph graph:

```
:::diagram An Agent Loop as a Graph
grid 1.5 0.9
node start "START" 0 0 round
node plan "PLAN" 1 0
node act "ACT\n(TOOLS)" 2 0
node check "DONE?" 3 0 diamond w=0.8 h=1.1
node end "END" 4 0 round
node human "HUMAN\nREVIEW" 3 1 dash
edge start plan
edge plan act
edge act check
edge check end "YES"
edge check plan "NO: REPLAN" bend=0.45
edge check human "UNSURE" dash
edge human plan "EDITS" bend=-0.3
loop act "TOOL CALL" bottom
:::
```



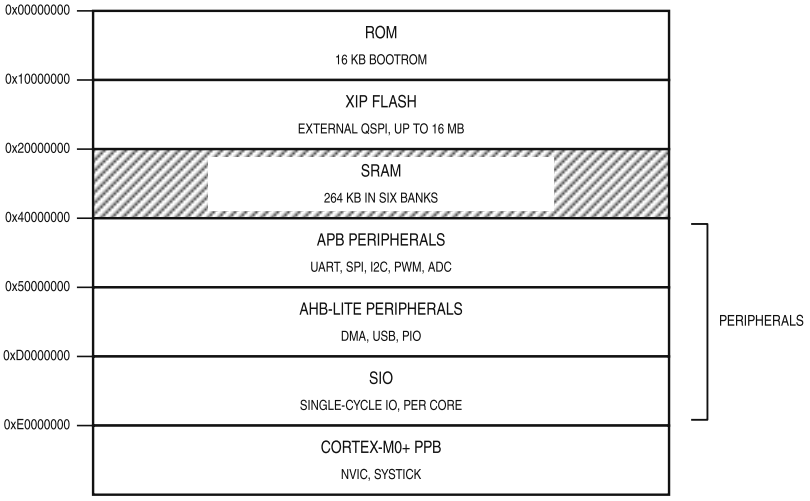
APO-509A

Figure A-9. An Agent Loop as a Graph

A.6.2 Stacks and memory maps.

`:::stack` sets layers top to bottom: a protocol stack, memory tiers, or with `at=` an address map. `span` brackets a run of layers. The RP2040's address map:

```
:::stack RP2040 Address Map
layer "ROM" "16 KB BOOTROM" at=0x00000000
layer "XIP flash" "EXTERNAL QSPI, UP TO 16 MB" at=0x10000000
layer "SRAM" "264 KB IN SIX BANKS" hatch at=0x20000000
layer "APB peripherals" "UART, SPI, I2C, PWM, ADC" at=0x40000000
layer "AHB-Lite peripherals" "DMA, USB, PIO" at=0x50000000
layer "SIO" "SINGLE-CYCLE IO, PER CORE" at=0xD0000000
layer "Cortex-M0+ PPB" "NVIC, SYSTICK" at=0xE0000000
span 4 6 "PERIPHERALS"
:::
```



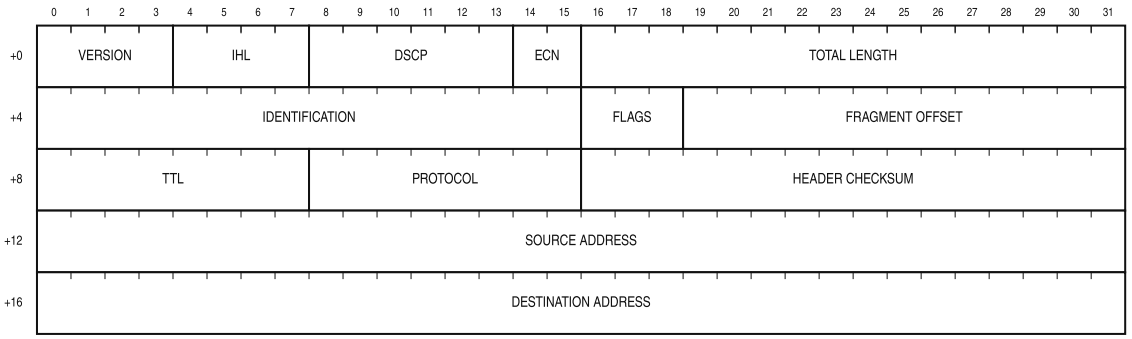
APO-510A

Figure A-10. RP2040 Address Map (Datasheet 2.2)

A.6.3 Bit fields.

`:::bits` lays out a register or a packet header, wrapping fields into rows of `width` bits; `numbers up` counts from 0 at the left as the RFCs do, `numbers down` from the top bit as a datasheet's register does; `hatch` marks reserved bits.

```
:::bits IPv4 Header
numbers up
field "Version" 4
field "IHL" 4
field "DSCP" 6
field "ECN" 2
field "Total length" 16
field "Identification" 16
field "Flags" 3
field "Fragment offset" 13
field "TTL" 8
field "Protocol" 8
field "Header checksum" 16
field "Source address" 32
field "Destination address" 32
:::
```

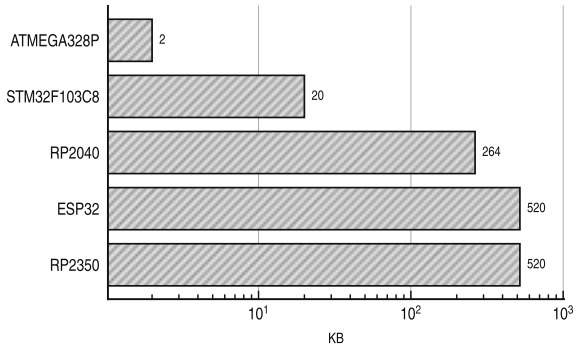
APO-511A

Figure A-11. IPv4 Header (RFC 791; DSCP and ECN from RFCs 2474 and 3168)

A.6.4 Bars.

`:::bars` compares categories: one value a series, series side by side or `stacked`, told apart by hatch with a key above only when there is more than one; `horizontal` for long names, `log` for spans of decades. Values print at the bar ends.

```
:::bars On-Chip SRAM
horizontal
log
unit "KB"
bar "ATmega328P" 2
bar "STM32F103C8" 20
bar "RP2040" 264
bar "ESP32" 520
bar "RP2350" 520
:::
```

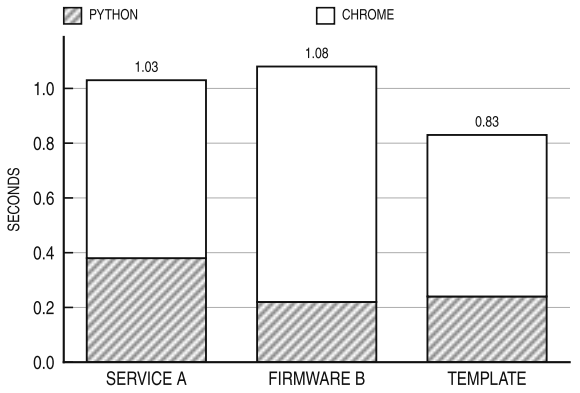


APO-512A

Figure A-12. On-Chip SRAM, KB

```
:::bars Time to Render a Sheet
series "Python" "Chrome"
stacked
unit "seconds"
bar "Service A" 0.38 0.65
bar "Firmware B" 0.22 0.86
bar "Template" 0.24 0.59
:::
```

Measured 10.28.2026, the best of three runs: Python is `render()`, Chrome the print to PDF.



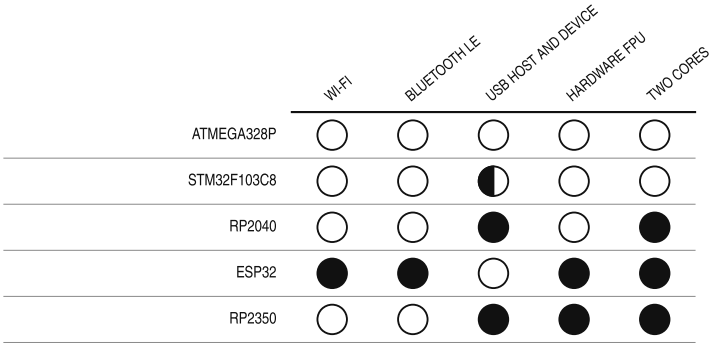
APO-513A

Figure A-13. Time to Render a Sheet, Seconds

A.6.5 Matrices.

`:::matrix` compares at a glance: `yes`, `part` and `no` print as a full, half and open dot, `-` as a dash, a number as a dot of that area, anything else as lettering.

```
:::matrix Five Microcontrollers
cols "Wi-Fi" "Bluetooth LE" "USB host and device" "Hardware FPU" "Two cores"
row "ATmega328P" no no no no no
row "STM32F103C8" no no part no no
row "RP2040" no no yes no yes
row "ESP32" yes yes no yes yes
row "RP2350" no no yes yes yes
:::
```



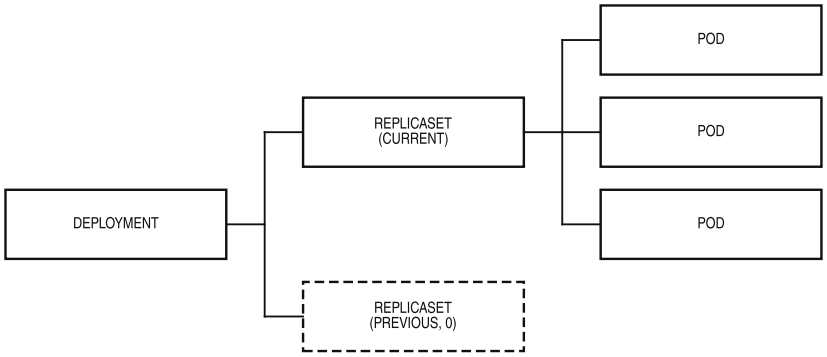
APO-514A

Figure A-14. Five Microcontrollers (Half Dot: the STM32F103's USB Is Device Only)

A.6.6 Trees.

`:::tree` draws a hierarchy from an indented outline, two spaces a level, the root at the left (`down` on the first line puts it at the top). Who owns what in a Kubernetes Deployment:

```
:::tree Kubernetes Ownership
Deployment
  ReplicaSet\n(current)
    Pod
    Pod
    Pod
  ReplicaSet\n(previous, 0) [dash]
:::
```



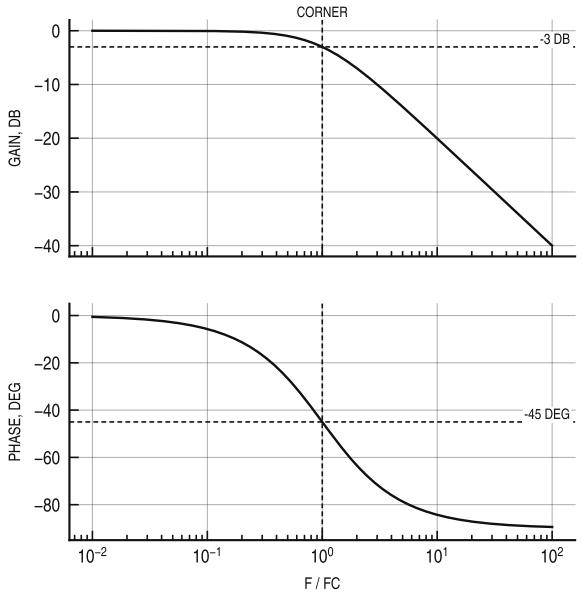
APO-515A

Figure A-15. A Deployment's Ownership Chain

A.6.7 Panels.

A `panel` line in a `:::plot` starts another panel under the first, on the same x: a Bode plot's magnitude and phase, here a first-order low-pass:

```
:::plot First-Order Low-Pass
range 0.01 100
log x
fn "-10*log10(1+x**2)"
ylabel "gain, dB"
hline -3.01 "-3 DB"
mark 1 "CORNER"
panel
fn "-degrees(arctan(x))"
ylabel "phase, deg"
ylim -95 5
hline -45 "-45 DEG"
xlabel "f / fc"
:::
```



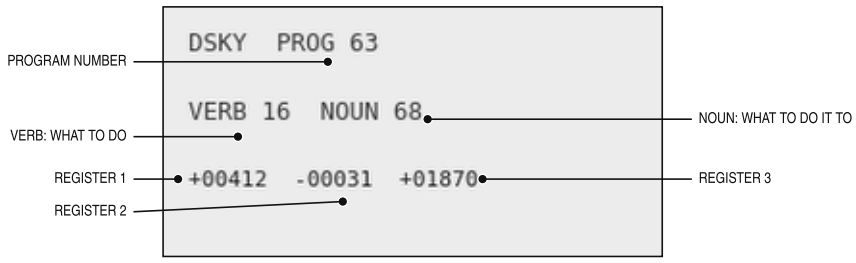
APO-516A

Figure A-16. First-Order Low-Pass, Gain and Phase

A.6.8 Callouts.

`:::callouts` names the parts of a picture: a leader from each point (fractions of the picture from its top left) to lettering in the margin on the nearer side. `gray` inverts a dark screen to ink.

```
:::callouts A Display Capture
image ../kitchen-sink/screen.png gray
at 0.33 0.22 "program number"
at 0.15 0.52 "verb: what to do"
at 0.53 0.45 "noun: what to do it to"
at 0.03 0.69 "register 1"
at 0.36 0.78 "register 2"
at 0.64 0.69 "register 3"
:::
```



APO-517A

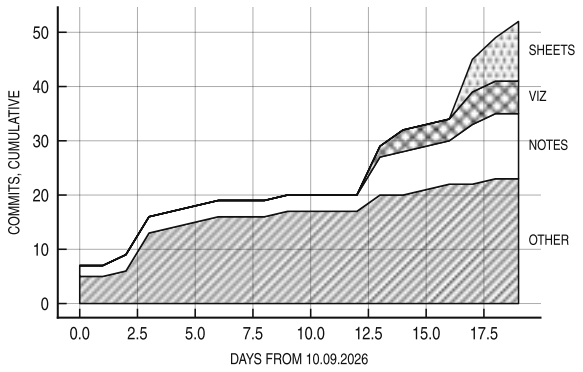
Figure A-17. A Display Capture, Named

A.7 DATA, FLOWS AND DECISIONS.

A.7.1 Stacked areas and shares.

`area` on a plot series stacks it as a hatched band on the area series before it; `percent` turns a panel's bands into shares of their sum. The commits to an example repository, by where they landed (a commit touching two areas counts in each):

```
:::plot Commits by Area
csv data/commits.csv
x day "days from 10.09.2026"
y other "OTHER" area
y notes "NOTES" area
y viz "VIZ" area
y sheets "SHEETS" area
ylabel "commits, cumulative"
:::
```

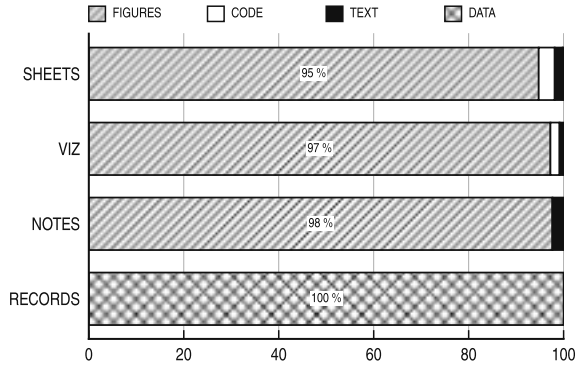


APO-518A

Figure A-18. Commits to an Example Repository by Area, Cumulative

`percent` on `:::bars` does the same for categories: each bar is its own 100 %. The bytes in four directories of the same repository, by kind of file (KB, tracked files, 10.28.2026):

```
:::bars Bytes by Kind
series "Figures" "Code" "Text" "Data"
percent
horizontal
bar "sheets" 2422 86 43 5
bar "viz" 2489 49 22 0
bar "notes" 1001 0 24 0
bar "records" 0 0 0 98
:::
```



APO-519A

Figure A-19. Bytes by Kind of File, Four Directories

A.7.2 Scatter and bubbles.

`:::scatter` places points, lettered if named, as bubbles if given a size; `fit` draws the least-squares line and its r, on the axes as drawn. Clock against on-chip SRAM:

```
:::scatter Clock and SRAM
xlabel "max clock, MHz"
ylabel "SRAM, KB"
log y
fit
point 20 2 "ATmega328P"
point 72 20 "STM32F103C8"
point 133 264 "RP2040"
point 240 520 "ESP32"
point 150 520 "RP2350"
:::
```

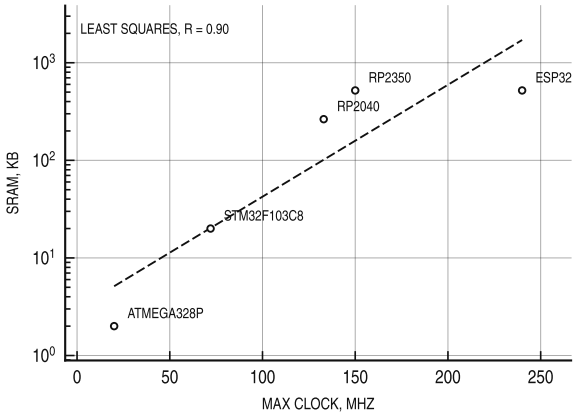


Figure A-20. Five Microcontrollers: Clock and SRAM

A.7.3 Distributions.

`:::dist` draws boxes from five numbers, or from raw CSV values (`data`, quartiles, whiskers to 1.5 IQR, the rest as outliers), with `violin` round them; `hist` draws a histogram instead. The time `render()` takes, 40 runs on each of three sheets, 10.28.2026; across all 120 the median is 237 ms and the 95th percentile 371 ms:

```
:::dist Render Time by Sheet
data "report A" data/render-ms.csv ms
where=sheet:report-a
data "report B" data/render-ms.csv ms
where=sheet:report-b
data "template" data/render-ms.csv ms
where=sheet:template
violin
unit "ms"
mark 371 "P95"
:::

:::dist All 120 Runs
hist data/render-ms.csv ms bins=24
unit "ms"
mark 237 "P50"
mark 371 "P95"
:::
```

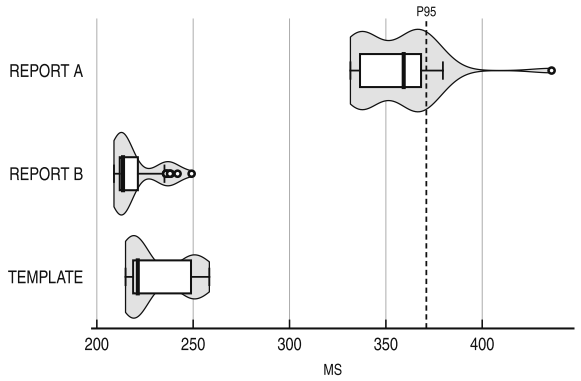
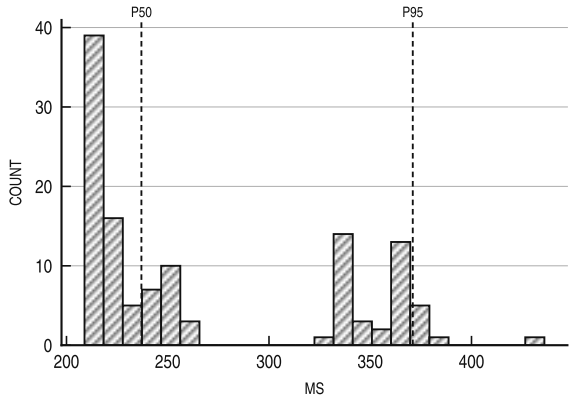


Figure A-21. Render Time by Sheet, 40 Runs Each



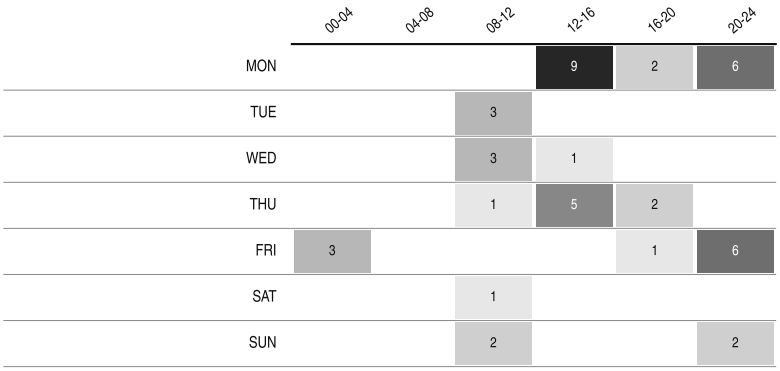
APO-522A

Figure A-22. All 120 Runs

A.7.4 Heatmaps.

`shade` on a `:::matrix` tints each numeric cell by its value and letters it: a heatmap in one ink. The 47 commits to the example repository by weekday and four-hour block, in the committer's local time:

```
:::matrix Commits by Hour
shade
cols "00-04" "04-08" "08-12" "12-16" "16-20" "20-24"
row "Mon" 0 0 0 9 2 6
row "Tue" 0 0 3 0 0 0
row "Wed" 0 0 3 1 0 0
row "Thu" 0 0 1 5 2 0
row "Fri" 3 0 0 0 1 6
row "Sat" 0 0 1 0 0 0
row "Sun" 0 0 2 0 0 2
:::
```



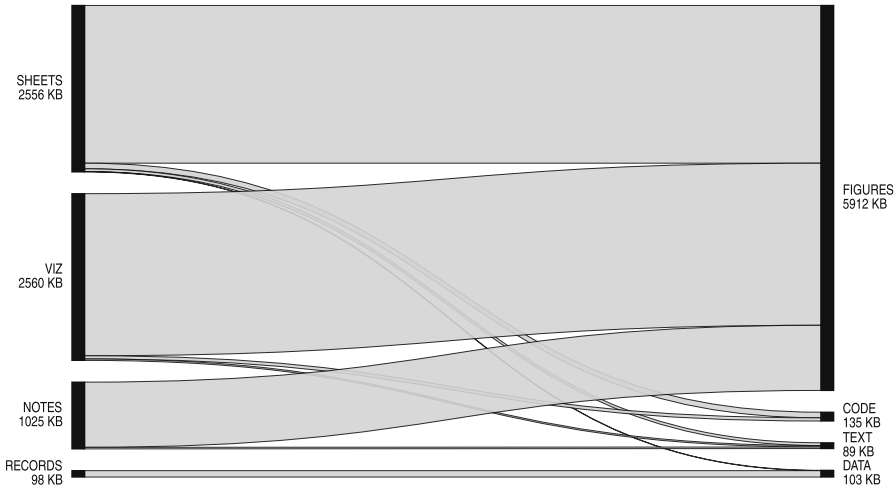
APO-523A

Figure A-23. Commits to the Example Repository by Weekday and Hour

A.7.5 Sankey.

`:::sankey` draws flows between stages, each ribbon as wide as its value; columns follow from the flows. The same bytes as Figure A-19, from directory to kind of file:

```
:::sankey Where the Bytes Are
unit "KB"
flow "sheets" "Figures" 2422
flow "sheets" "Code" 86
flow "sheets" "Text" 43
flow "sheets" "Data" 5
flow "viz" "Figures" 2489
flow "viz" "Code" 49
flow "viz" "Text" 22
flow "notes" "Figures" 1001
flow "notes" "Text" 24
flow "records" "Data" 98
:::
```



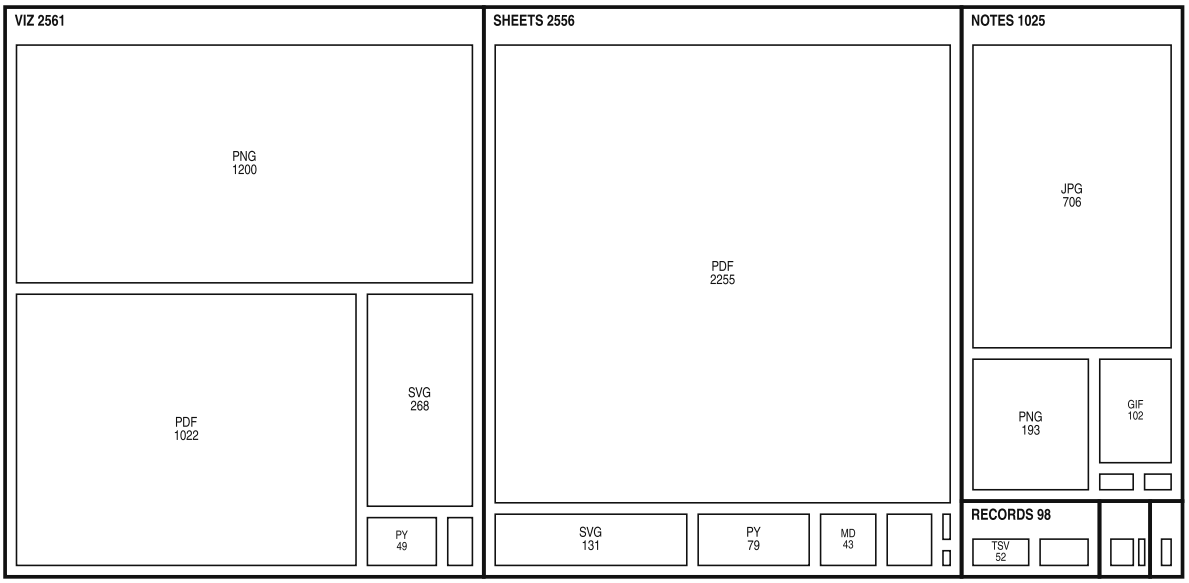
APO-524A

Figure A-24. Where the Bytes Are, Directory to Kind of File (KB)

A.7.6 Treemaps.

`:::treemap` sets nested shares as rectangles from an indented outline; a leaf ends in its value. The example repository's tracked files by directory and extension, in KB:

```
:::treemap Disk Use
sheets
  pdf 2255
  svg 131
  py 79
...
notebooks
  ipynb 23
:::
```



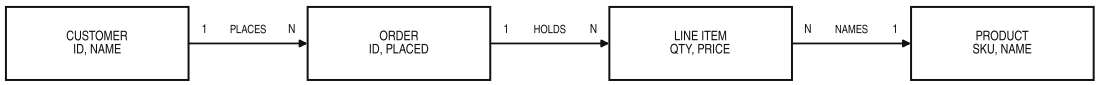
APO-525A

Figure A-25. Tracked Files by Directory and Extension, KB (10.28.2026)

A.7.7 Entity relationships.

`ends=1:N` on a diagram edge letters each end: the cardinality of an ER model. An example schema:

```
:::diagram An Order Schema
grid 1.9 0.8
node cust "CUSTOMER\nid, name" 0 0
node order "ORDER\nid, placed" 1 0
node line "LINE ITEM\nqty, price" 2 0
node prod "PRODUCT\nsku, name" 3 0
edge cust order "PLACES" ends=1:N
edge order line "HOLDS" ends=1:N
edge line prod "NAMES" ends=N:1
:::
```



APO-526A

Figure A-26. An Order Schema (Example)

A.7.8 Quadrants, sets and funnels.

`:::quadrant` names the four quarters of two axes and places points on them, here one writer's estimate of the figures on this sheet; `:::venn` letters the regions of two or three sets, here the matrix of Figure A-14 seen another way; `:::funnel` narrows stage by stage, each with its share of the first: the example repository's files.

```
:::quadrant What to Draw Next
x "low effort" "high effort"
y "low value" "high value"
name 1 "Plan for"
name 2 "Do first"
name 3 "Maybe"
name 4 "Skip"
point 0.2 0.85 "bars"
point 0.3 0.8 "heatmap"
point 0.62 0.7 "sankey"
point 0.85 0.3 "sunburst"
point 0.9 0.2 "exploded view"
point 0.35 0.35 "venn"
:::
```

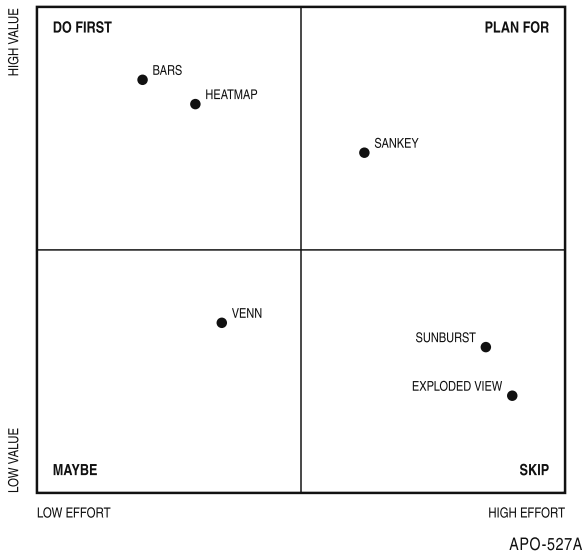


Figure A-27. Value and Effort of Figure Types (One Writer's Estimate)


```
:::venn Three Features
set "Wi-Fi" "USB host and device" "Two cores"
in ABC ""
in AC "ESP32"
in BC "RP2040\nRP2350"
in out "ATmega328P\nSTM32F103C8"
:::
```

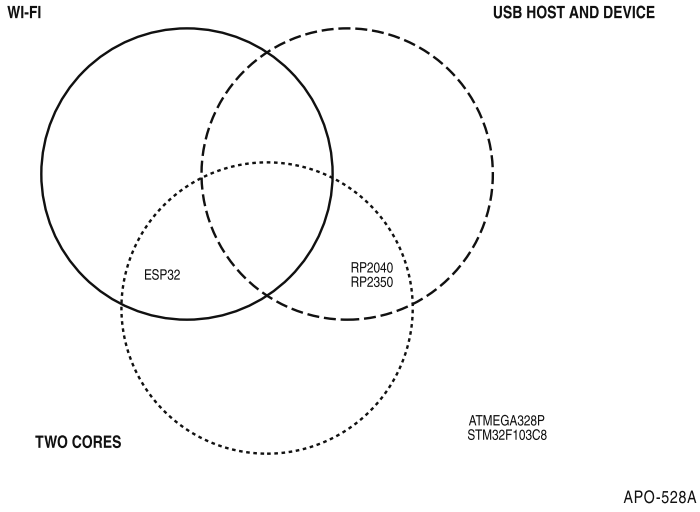
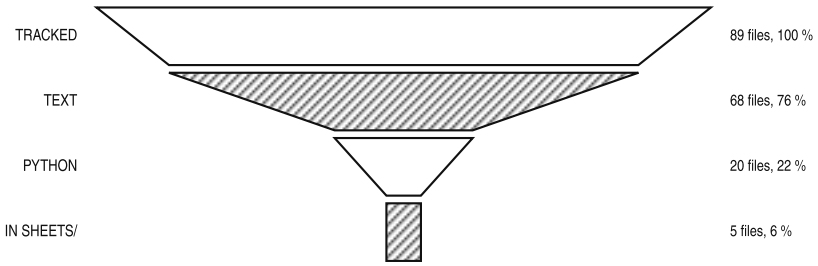


Figure A-28. Five Microcontrollers, Three Features (from Figure A-14)

```
:::funnel Files
unit "files"
stage "tracked" 89
stage "text" 68
stage "Python" 20
stage "in sheets/" 5
:::
```



APO-529A

Figure A-29. Files in the Example Repository, 10.28.2026

A.8 WHICH FIGURE.

Table A-2. Choosing a Figure

If the material is	Use	Not
a quantity over time, or a function of x	`:::plot` (`fn` for exact, `y` for data, `area` for parts)	bars per date
parts of a whole over time	`:::plot` with `area` and `percent`	a row of pies
categories compared	`:::bars`, `horizontal` for long names	a line through categories
parts of each category's whole	`:::bars percent`	pies, donuts
two quantities against each other	`:::scatter`, `fit` for the trend	a table
a spread, latency, tolerance	`:::dist`: box, violin, `hist`	a bare average
a grid of values: hour by day, x by y	`:::matrix shade`	a table of numbers alone
yes and no across items	`:::matrix` with `yes part no`	ticks in a table
used against capacity	`:::budget`	a pie
components and what feeds what	`:::diagram`	prose
decisions, loops, states	`:::diagram` with `diamond`, `bend=`, `loop`	a numbered list of ifs
tables and their relations	`:::diagram` with `ends=1:N`	
messages between actors over time	`:::sequence`	a diagram with numbered arrows
signals on a clock	`:::signals`	a table of levels
phases, a schedule, a mission	`:::timeline`	
a hierarchy	`:::tree`	indented text
nested shares: disk, budget by part	`:::treemap`	a sunburst
quantities moving between stages	`:::sankey`	arrows with numbers
stages that narrow	`:::funnel`	
layers, tiers, an address map	`:::stack`	
a register or a header	`:::bits`	a table of offsets
two judgements at once	`:::quadrant`	
overlap between groups	`:::venn` (three sets at most)	
the parts of a thing you can photograph	`:::callouts`	a long caption
a procedure	`:::checklist`	a paragraph of steps

Anything that must move, take input or recalculate (a planner with sliders, a live simulation) is not a sheet: it belongs in an interactive page. A sheet carries its static half: the figure at its stated values, and the table or formula behind it.

A.9 MAKING A SHEET.

TURN A REPORT INTO A SHEET — **WRITER**

```
1  python sheet.py report.md --png
     -s section=N -s tab=XXX -s date=MM.DD.YYYY
2  read every page PNG; fix what crowds
3  add figures, equations, cautions, checklists
4  byline DRAFT; send to the fact checker
5  apply corrections; name the checker
6  commit; send the PDF
```

For a sheet of your own, copy `template.md`: it has the front matter and one of each block, with comments that do not print. Later changes go in `:::changed` with `change:` set in the front matter; the basic date stays.

CAUTION

Chrome does not print into the page margins, so the gutter numbers, edge tab and change bars sit inside the content box: keep `main`'s padding when restyling. A block closes at the first line of exactly its own colons, so a block that holds blocks takes four (`::::changed` around `:::equations`), and a row's source cannot be shown in a row. Wide figures (`bits`, `signals`, `sankey`, `treemap`, `timeline`, a heatmap) want the full column: in a `::::row` their lettering shrinks below reading size.